

Temporal Network Embedding Enhanced With Long-Range Dynamics and Self-Supervised Learning

Zhizheng Wang¹, Yuanyuan Sun¹, *Member, IEEE*, Zhihao Yang¹, *Senior Member, IEEE*,
Liang Yang¹, *Member, IEEE*, and Hongfei Lin¹, *Member, IEEE*

Abstract—Temporal network embedding (TNE) has promoted the research of knowledge discovery and reasoning on networks. It aims to embed vertices of temporal networks into a low-dimensional vector space while preserving network structures and temporal properties. However, most existing methods have limitations in capturing dynamics over long distances, which makes it difficult to explore multihop topological associations among vertices. To tackle this challenge, we propose LongTNE, which learns the long-range dynamics of vertices to endow TNE with the ability to capture high-order proximity (HP) of networks. In LongTNE, we employ graph self-supervised learning (Graph SSL) to optimize the establishment probability of deep links in each network snapshot. We also present an accumulated forward update (AFU) module to fathom global temporal evolution among multiple network snapshots. The empirical results on six temporal networks demonstrate that, in addition to achieving state-of-the-art performance on network mining tasks, LongTNE can be handily extended to existing TNE methods.

Index Terms—Graph self-supervised learning (Graph SSL), long-range dynamics, temporal network embedding (TNE).

I. INTRODUCTION

WITH the development of network representation learning (NRL), knowledge discovery and reasoning on networks have been active research for decades [1], [2], [3], [4], [5], [6], [7], which is involved into social networks [8], [9], road networks [10], spatial networks [11], and visual networks [12], [13]. NRL aims to learn a mapping function $\mathcal{L}$ to project vertex v into a d -dimensional vector. They typically discover implicit knowledge through link prediction [14], [15]. Despite the advanced NRL algorithms having achieved promising performance in many network-mining scenarios, they are confronted with the challenges of discovering long-range and evolutionary knowledge from temporal networks.

Manuscript received 22 December 2021; revised 20 August 2022 and 1 March 2024; accepted 30 March 2024. Date of publication 15 April 2024; date of current version 1 March 2025. This work was supported in part by the National Key Research and Development Program of China under Grant 2022YFC3301801, in part by the Fundamental Research Funds for the Central Universities under Grant DUT22ZD205, and in part by the Major Science and Technology Projects of Yunnan Province under Grant 202002ab080001-1. (Corresponding author: Yuanyuan Sun.)

Zhizheng Wang was with the the School of Computer Science and Technology, Dalian University of Technology, Dalian 116024, China. He is now with the National Center for Biotechnology Information (NCBI), National Library of Medicine (NLM), National Institutes of Health (NIH), Bethesda, MD 20894 USA (e-mail: wzz0727@gmail.com).

Yuanyuan Sun, Zhihao Yang, Liang Yang, and Hongfei Lin are with the School of Computer Science and Technology, Dalian University of Technology, Dalian 116024, China (e-mail: wzz0727@gmail.com; syuan@dlut.edu.cn; yangzh@dlut.edu.cn; liang@dlut.edu.cn; hflin@dlut.edu.cn).

This article has supplementary downloadable material available at <https://doi.org/10.1109/TNNLS.2024.3384348>, provided by the authors.

Digital Object Identifier 10.1109/TNNLS.2024.3384348

2162-237X © 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.

See <https://www.ieee.org/publications/rights/index.html> for more information.

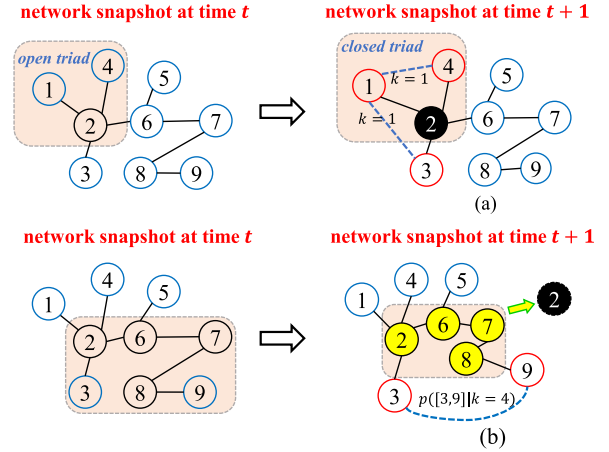


Fig. 1. Sample of (a) triad closure process and (b) long-range dynamic proximity. The triad closure process aims to establish an edge for two one-step reachable vertices while the long-range dynamic proximity aims to establish an edge for two k -step ($k \geq 1$) reachable vertices.

Practically, early NRL methods focused on static networks [16], [17], [18], [19], [20] and encoded networks by aggregating stable topology of vertices. However, networks exhibit complex temporal and spatial properties in the real world [21], so their topological structure will be constantly evolving. As a result, temporal network embedding (TNE) is proposed to capture dynamic features of networks [22], [23], [24], [25], [26], [27], [28], [29], [30] in recent years. They either simplify the evolution process as a series of network snapshots or explore the evolution pattern of edges via the temporal point process [31]. However, most TNE methods only model the local topology through stochastic processes, which indicates they emphasize learning short-range dynamics of vertices. However, in temporal networks, vertices will gradually show similarities over long distances as time goes on. Therefore, although TNE provides an effective way of learning the dynamic representation of networks, they still have limitations in discovering deep knowledge from networks.

Naturally, the above challenges can be tackled by a scalable TNE method that can simultaneously explore networks' deep and evolutionary patterns by modeling long-distance proximity. Among the existing TNEs, one of the illuminating studies in preserving distance approximation is DynamicTriad [23]. As Fig. 1(a) shows, it modeled the triad closure process that how a closed triad develops from an open triad. Essentially, DynamicTriad aims to generate an edge for two unconnected vertices but with common neighborhoods, which is an application of local structural evolution [32], [33], [34]

in encoding path information. Inspired by DynamicTriad, we extend the triad closure process to long-path reachable vertices, defined as *long-range dynamic proximity* in our study. As shown in Fig. 1(b), two four-step ($k = 4$) reachable vertices (e.g., vertices 3 and 9) and a group of internal vertices (e.g., vertices 2, 6, 7, and 8) construct a pseudo-closed triad, from which we can capture the four-hop dynamic features between vertices 3 and 9.

However, applying the triad closure process to preserve long-range dynamics of vertices may have the following problems.

- 1) How do we discriminate topological roles of unconnected vertices? Methods based on local structure in developing a triad closure considered all unobserved links as negative samples and maximized the distance between positive and negative samples. So, they failed to distinguish the topological roles between long-path dependent vertices and irrelevant vertices.
- 2) How do we simulate the triad closure process in temporal networks with the adaptive strategy? Generally, the probability of generating links for two unconnected vertices was estimated by static parameters derived from the network topology. However, this probability should be adjusted based on the distance among vertices in path-dependent scenarios.
- 3) Which training manner should be selected to enable the model scalability? Recent attention-based graph neural networks [35], [36], [37] also attempted to improve the triad closure process for exploring high-order proximity (HP). They were typically trained in a supervised fashion and required a large number of annotated data. However, due to potentially connected vertices being uncertain and numerous in temporal networks, it is impractical to keep track of newly emerging data manually.

Self-supervised learning (SSL) provides a meaningful solution to address the above issues. Recently, SSL has achieved promising improvement in learning graph representation [38], [39], [40], [41], [42] via automatically exploring augmentations for unlabeled graph data. Therefore, combining SSL with graph distance proximity, we propose LongTNE in this work, a fine-tuning method that enhances temporal network embedding by exploring the **long-range** dynamics of vertices. First, LongTNE uses the graph traversal method to establish the hierarchical context for vertices, distinguishing the long-path-dependent vertices from irrelevant vertices. Afterward, LongTNE actively learns an adaptive weight matrix to assign dynamic similarity weights to the long-path reachable vertices. Next, LongTNE introduces the graph self-supervised learning (Graph SSL) strategy to construct augmented training samples from hierarchical topology and fine-tune the pretrained vertex embeddings. Finally, LongTNE integrates an accumulated forward update (AFU) module to acquire global temporal evolution based on the assumption that a network will evolve smoothly over time.

Collectively, several noteworthy contributions to our study can be summarized as follows.

- 1) We study the effectiveness of long-range dynamics in TNE and propose an extension algorithm LongTNE to

deduce deep and evolutionary knowledge from temporal networks.

- 2) In LongTNE, we design an augmentation strategy for unlabeled graph data, which combines Graph SSL and deep network topology to fine-tune the pretrained vertex embeddings.
- 3) We present a time-step partition schema to prepare network snapshots for LongTNE and elucidate the effectiveness of LongTNE with extensive empirical experiments on large-scale temporal networks.

II. RELATED WORK

A. Temporal Network Embedding

Currently, the TNE methods can be categorized into discrete-time TNE and continuous-time TNE. Dynamic network embedding (DNE) [22] is presented to encode temporal networks under the assumption that a network will not change much during a short time. Learning representations over dynamic graphs (DyRep) [43] defined the temporal evolution over networks as an association and communication process. Hawkes process based temporal network embedding (HTNE) [44] was a Hawkes process-based method that modeled the neighborhood formation sequence in continuous time. Dynamic co-embedding model for temporal attributed networks (DCTANs) [29] learned temporal network representations by integrating network topology and vertex attributes. Other methods [7], [45], [46], [47] emphasized learning the dynamic features by combining the global or local topology with the temporal formation of edges. In addition, with the in-depth study of temporal evolution patterns, many neural network models have also been applied to learn the representations of temporal networks [30], [48], [49], [50], [51].

B. High-Order Proximity Preservation

The HP of a network is usually computed by the k th power of its normalized adjacency matrix or is approximated by a random walk [52]. To encode static networks, learning graph representations with global structural information (GraRep) [53] modeled the high-order neighbors of those vertices with small degrees. HP preserved embedding (HOPE) [54] preserved the asymmetric transitivity of networks by approximating HP. To encode temporal networks, dynamic HP preserved embedding (DHPE) [25] adopted the generalized singular value decomposition (SVD) to preserve HP for learning dynamic features of networks. High-order nonlinear information preserving (HNIP) [55] proposed a deep-guided autoencoder to capture the highly nonlinear structure while preserving the temporal proximity of vertices. Furthermore, methods such as hybrid-order representation learning network (HORLN) [11] also explored the HP of spatial data in the anomaly detection task.

C. Graph Self-Supervised Learning

Graph SSL indicates the learning paradigm where supervision signals (or pseudo-labels) are generated from graph data. Some studies explore different graph augmentations to learn representations of static networks [56] and be applied to different fields [57], [58], [59]. Self-supervised graph-level representation learning with local and global structure

(GraphLoG) [38] is a unified framework for self-supervised whole-graph representation learning, which preserves the local-instance and global-semantic structure of networks. Graph adversarial self-supervised learning (GASSL) [39] automatically generated challenging negative samples by adding perturbations to the input and were adversarially trained for the encoder. Graph contrastive coding (GCC) [60] developed a self-supervised GNN pretrained framework to capture the universal network topological properties across multiple networks. Graph contrastive learning (GraphCL) [61] designed four types of graph augmentations to incorporate various priors and verified the robustness of the self-supervised training fashion. Cross reconstruction transformer (CRT) [42] introduced SSL in time-series representation learning, which was tailored to the cross-domain dropping-reconstruction task.

III. PRELIMINARY KNOWLEDGE

In this section, we give a brief preliminary knowledge of TNE, HP, and Graph SSL.

Definition 1 (Temporal Network): A temporal network consists of a sequence of timestamped edges, where each edge is established at a certain time to connect two vertices. Formally, a temporal network is defined as $G_T = (\mathcal{V}, \mathcal{E}, \mathcal{A}, \mathcal{T})$ that consists of vertex set $\mathcal{V}$ and edge set $\mathcal{E} \subset \mathcal{V} \times \mathcal{V}$. $\mathcal{A}$ denotes the adjacency matrix and $\mathcal{T}$ denotes the temporal sequence of edges. Given a G_T , it can be divided into t snapshots according to the temporal sequence $\mathcal{T}$. The t th snapshot $G_t = (\mathcal{V}_t, \mathcal{E}_t, \mathcal{A}_t)$ contains its vertex set $\mathcal{V}_t$, edge set $\mathcal{E}_t$, and the adjacency matrix $\mathcal{A}_t$.

Definition 2 (TNE): Given a set of network snapshots $\{G_1, \dots, G_t\}$, TNE aims to learn a mapping function $\mathcal{L} : \mathcal{V}_t \rightarrow \mathbb{R}^{|\mathcal{V}_t| \times d}$ to obtain a d -dimensional representation $\mathbf{X}_t^E$ for G_t , where $d \ll |\mathcal{V}_t|$. A general learning framework for TNE is written as follows [50]:

$$\mathbf{X}_t^E = g(\mathcal{L}(\mathcal{A}_t, \mathbf{X}_t), \mathbf{X}_{t-1}^E) \quad (1)$$

where $\mathbf{X}_t$ is the initial (pretrained) vertex embeddings of G_t . $\mathbf{X}_{t-1}^E$ denotes the representations of the latest historical network snapshot G_{t-1} . g is a global function that captures evolving regularities among different network snapshots.

Definition 3 (HP): HP can be generalized to k -order proximity based on the random walk, that is, the probability that a random walk starts at vertex u and reaches vertex v in k steps. After sampling the vertex sequence, the k th power of the *first-order* proximity is usually used to approximate the HP, as shown in the following equation:

$$H(u, v) = \left(\frac{1}{1 + \exp(-(\vec{u})^\top \cdot \vec{v})} \right)^k \quad (2)$$

where $\vec{u}$ and $\vec{v}$ stand for the embeddings of vertices u and v , respectively.

Definition 4 (Graph SSL): For an input network snapshot $G_t = (\mathcal{V}_t, \mathcal{E}_t, \mathcal{A}_t)$, let $\mathcal{D}_t$ denote uniform distributions over samples (edges) in G_t . Graph SSL attempts to learn a graph encoder $\mathcal{F}^*$ via utilizing information from $\mathcal{D}_t$ and minimizing the self-supervised loss $\mathcal{O}_{\text{ssl}}$, which is formulated as follows:

$$\mathcal{F}^* = \arg \min_{\mathcal{F}, [\mathcal{P}]} \mathcal{O}_{\text{ssl}}(\mathcal{F}, [\mathcal{P}], \mathcal{D}_t) \quad (3)$$

where $\mathcal{D}_t$ typically satisfies $(\mathcal{V}_t, \mathcal{E}_t) \sim \mathcal{D}_t$ in the unlabeled graph G_t . After obtaining the trained encoder $\mathcal{F}^*$, the generated representations can be adapted to various downstream tasks via an optional pretext decoder $\mathcal{P}$.

A. Problem Formulation

The problem studied in our work can be formally defined as (4), which aims to learn a low-dimensional representation of the relational network with temporal attributes and thus discovering profound and evolutionary knowledge from networks. Specifically, we attempt to learn an enhanced mapping function $\mathcal{L}^*$ that stimulates HP of the current network snapshot $G_t = (\mathcal{V}_t, \mathcal{E}_t, \mathcal{A}_t)$ by exploiting the representations learned from the latest historical network snapshot G_{t-1} . To obtain the scalable $\mathcal{L}^*$, we use the Graph SSL as the optimizer to construct augmented data $\mathcal{D}_t$ for G_t . Based on $\mathcal{D}_t$, we design an auxiliary pretext decoder ψ to enable the pretrained mapping function $\mathcal{L}$ to capture HP of G_t

$$\begin{aligned} \mathbf{X}_t^E &= g(\mathcal{L}^*(\mathcal{A}_t, \mathbf{X}_t), \mathbf{X}_{t-1}^E) \\ \text{w.r.t } \mathcal{L}^* &= \arg \min_{\mathcal{L}, \psi} \mathcal{O}_{\text{ssl}}(\mathcal{L}, \psi, \mathcal{D}_t). \end{aligned} \quad (4)$$

IV. METHODOLOGY

The detailed framework of LongTNE is illustrated in Fig. 2. As sketched in Fig. 2(a), LongTNE tackles the problem formulated in (4) through a two-step fine-tuning paradigm. The embedding stream of LongTNE contains two parts: One is the *long-distance dynamic proximity preserved with SSL* (LDPP-SSL) that aims to learn the enhanced mapping function $\mathcal{L}^*$. It contains the long-distance dynamic proximity preservation (LDPP) module and the Graph SSL module. The other is the AFU that is used as the global function g to capture evolving regularities of the temporal network.

A. Long-Distance Dynamic Proximity Preservation

As described in Section I, distinguishing the topological roles of vertices and adapting to the dynamic similarity among vertices are two essential problems on the road to preserving long-range dynamics. Therefore, we present LDPP as shown in Fig. 2(b). It first constructs the hierarchical topology for vertices and then learns an adaptive weight matrix within long-path reachable vertices. At last, it employs a variation of the HP function to preserve the long-range dynamics.

1) Hierarchical Topology Construction: Considering temporal networks are directed in most realistic scenarios, we enforce direction constraints on edges to obtain the directed network snapshot $G_t^D = (\mathcal{V}_t, \mathcal{E}_t^{\rightarrow}, \mathcal{A}_t^{\rightarrow})$. In G_t^D , the path between two vertices is limited from the head vertex to the tail vertex, compressing the storage memory of adjacency matrix $\mathcal{A}_t^{\rightarrow}$ by about 50%. After that, we apply the *depth-first search* (DFS) on G_t^D to construct the hierarchical context $\mathbb{T}$ (aka long-distance traversal tree) for each head vertex, which can be denoted as follows:

$$\mathbb{T}(\mathbb{V}, \mathbb{E}) = \text{DFS}(G_t^D, r) \quad (5)$$

where $\mathbb{V}$ and $\mathbb{E}$ denote vertices and edges in $\mathbb{T}$, respectively. $r \in \mathcal{V}_t$ represents the root vertex in $\mathbb{T}$. Notably, each root vertex will correspond to a long-distance traversal tree.

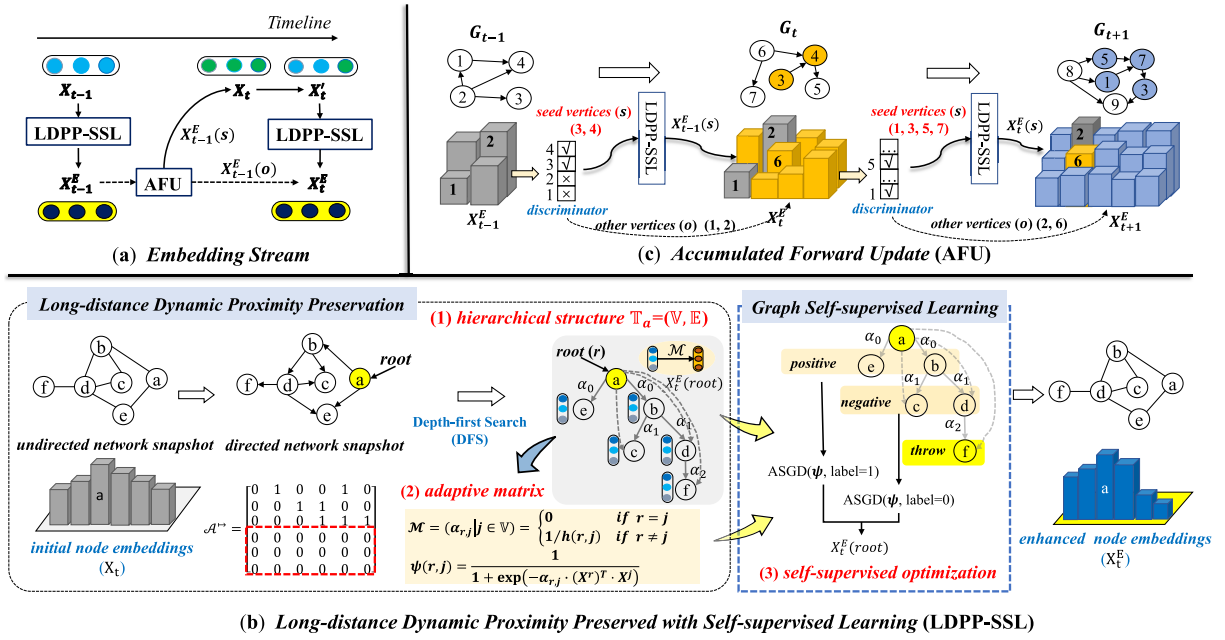


Fig. 2. Overall framework of LongTNE. (a) Streaming of vertex embeddings in LongTNE. $\mathbf{X}_t$ denotes the initial vertex embeddings of G_t . $\mathbf{X}_t^E$ denotes the new initial vertex embeddings of G_t . $\mathbf{X}_t^E$ denotes the enhanced vertex embeddings of G_t . (b) Example of the LDPP-SSL module. $\mathcal{M}$ denotes the adaptive proximity matrix. ASGD denotes the asynchronous stochastic gradient descent. (c) Flow of the AFU module.

As shown in Fig. 2(b), $\mathbb{T}$ is a connected subgraph starting from the root vertex r and has explored the deepest reachable vertices of r based on the network topology. Unlike the random walk strategy, there are no duplicate vertices in $\mathbb{V}$, so the long-path dependent and unrelated vertices can be distinguished. In $\mathbb{T}$, the distance from the context vertex ($\mathbb{V} \setminus r$) to the root vertex will gradually increase as the hierarchy goes deep.

2) *Adaptive Weight Matrix*: Based on the hierarchical structure implied in each $\mathbb{T}$, the dynamic probability of establishing an edge between the root vertex and other vertices can be estimated by their distances (or hops). The adaptive weight matrix learned from $\mathbb{T}$ can be formally represented as follows:

$$\mathcal{M} = (\alpha_{r,j} | j \in \mathbb{V}) = \begin{cases} 0, & \text{if } r = j \\ \frac{1}{h(r,j)}, & \text{if } r \neq j \end{cases} \quad (6)$$

where $h(\cdot)$ is a distance function indicating the number of hops between vertex $j \in \mathbb{V} \setminus r$ and the root vertex r . $\alpha_{r,j}$ represents the similarity weight between r and j . Essentially, the matrix $\mathcal{M}$ is used to capture the dynamic similarity between two long-path reachable vertices.

3) *Long-Range Dynamic Preservation Function*: Due to the operation of the k th power, it is time-consuming to preserve long-range dynamics with (2). Therefore, we make a variation based on $\mathbb{T}$ and $\mathcal{M}$ to approximate the k th power operation, which is defined as the following weighted *first-order* proximity function:

$$\begin{aligned} \psi(r, j) &= \frac{1}{1 + \exp(-f(\mathbb{X}^r, \mathbb{X}^j))} \\ &= \frac{1}{1 + \exp(-\alpha_{r,j} \cdot (\mathbb{X}^r)^\top \cdot \mathbb{X}^j)} \end{aligned} \quad (7)$$

where f denotes the weighted-bitwise multiplication between embeddings of the root vertex r and vertex j . $\mathbb{X} \in \mathbb{R}^{|\mathbb{V}| \times d}$ is the d -dimensional embedding matrix of $\mathbb{V}$, which is obtained by the pretrained temporal mapping function $\mathcal{L}$.

Equation (7) constructs a new distribution $\psi(\cdot, \cdot)$ over the network space $\mathbb{V} \times \mathbb{V}$, containing the *first-order* edges $\mathbb{E}^{f-o} \subset \mathbb{E}$ and the *high-order* edges $\mathbb{E}^{h-o} \not\subset \mathbb{E}$. The function ψ integrates all edges to the *first-order* links by setting variable weights α to unify the calculation. In this way, we can minimize the KL-divergence (D_{kl}) between ψ and its empirical probability $\hat{\psi}(r, j) = (w_{r,j} / \sum_{(r,j) \in \mathcal{S}_t} w_{r,j})$ to preserve the long-range dynamics of vertices

$$\begin{aligned} O_{kl} &= \sum_{r \in \mathcal{V}_t} D_{kl}(\hat{\psi}(r, j) || \psi(r, j)) \\ &= \sum_{r \in \mathcal{V}_t} \sum_{(r,j) \in \mathcal{S}_t} \hat{\psi}(r, j) \cdot \log \frac{\hat{\psi}(r, j)}{\psi(r, j)} \\ &= \sum_{r \in \mathcal{V}_t} \sum_{(r,j) \in \mathcal{S}_t} \frac{w_{r,j}}{W} \log \frac{w_{r,j} (1 + \exp(-\alpha_{r,j} (\mathbb{X}^r)^\top \mathbb{X}^j))}{W} \\ &= \sum_{r \in \mathcal{V}_t} \left(\frac{1}{W} \sum_{(r,j) \in \mathcal{S}_t} w_{r,j} \log \left(\frac{1 + \exp(-\alpha_{r,j} (\mathbb{X}^r)^\top \mathbb{X}^j)}{1/\psi(r, j)} \right) \right) \\ &\quad + \sum_{r \in \mathcal{V}_t} \left(\frac{w_{r,j}}{W} \sum_{(r,j) \in \mathcal{S}_t} \log w_{r,j} - \frac{w_{r,j}}{W} \sum_{(r,j) \in \mathcal{S}_t} \log W \right) \end{aligned} \quad (8)$$

where $\mathcal{S}_t = \mathbb{E}^{f-o} \cup \mathbb{E}^{h-o}$ and $W = \sum_{(r,j) \in \mathcal{S}_t} w_{r,j}$. Notice that $w_{r,j} = 0$, if r and j have no immediate links in $\mathbb{T}$, that is, $(r, j) \in \mathbb{E}^{h-o}$. Otherwise, $w_{r,j} = 1$, if there is an edge between r and j , that is, $(r, j) \in \mathbb{E}^{f-o}$. Therefore, after omitting some constants in (8), we can obtain the enhanced mapping function $\mathcal{L}^*$ by calculating the following objective function:

$$\mathcal{L}^* = \text{minimize} \left(- \sum_{r \in \mathcal{V}_t} \sum_{(r,j) \in \mathcal{S}_t} w_{r,j} \cdot \log \psi(r, j) \right). \quad (9)$$

B. Graph Self-Supervised Learning

A tricky problem in (9) is that it models all vertex pairs related to the root vertex in $\mathbb{T}$. So, it is computationally expensive for large-scale and dense temporal networks. In addition, the small-world network theory has proved that it is challenging to establish edges for two vertices if their distance is too long [62], [63]. To this end, we select to facilitate computing the function ψ by introducing Graph SSL as the model optimizer.

First, we build the augmented supervision signals for the unlabeled temporal network. It is a uniform distribution ($\mathcal{D}_t$) over the samples $\mathcal{S}_t$ in (8). Specifically, we sample the vertices in $\mathbb{T}$ to reduce the number of edges involved in the calculation and introduce a hyperparameter θ to generate positive ($y = 1$) and negative links ($y = 0$) from the sampled vertices, which can be formally defined as follows:

$$\mathcal{D}_t = \begin{cases} (r, v^n), & y = 0 \text{ if } h(r, v^n) > \theta \\ (r, v^n), & y = 1 \text{ if } h(r, v^n) \leq \theta \end{cases} \quad (10)$$

where v^n denotes the sampled vertex and satisfies $v^n \sim P_n(j)$. $P_n(j) \propto \lceil (d_j)^{3/4} \rceil$ and d_j is the degree of vertex j .

Second, we take the long-range dynamic function ψ as the pretext decoder of the Graph SSL, in which we use $\mathcal{D}_t$ with pseudo-labels as the target data to optimize the decoder. Thus, the function ψ can be rewritten as follows:

$$\begin{aligned} \psi(r, v^n) &= \sigma(-f'(\mathbb{X}^r, \mathbb{X}^{v^n})) \\ f'(\mathbb{X}^r, \mathbb{X}^{v^n}) &= \begin{cases} (\mathbb{X}^r)^\top \cdot \mathbb{X}^{v^n}, & h(r, v^n) > \theta \\ \alpha_{r,v^n} \cdot (\mathbb{X}^r)^\top \cdot \mathbb{X}^{v^n}, & h(r, v^n) \leq \theta \end{cases} \end{aligned} \quad (11)$$

where σ represents the sigmoid function. $f'(\mathbb{X}^r, \mathbb{X}^{v^n})$ is the masked bitwise multiplication function between the root vertex r and the sampled vertex v^n .

Finally, we integrate (10) and (11) into the Graph SSL framework for endowing the pretrained mapping function $\mathcal{L}$ with the ability to capture the long-range dynamics of vertices. Therefore, the enhanced mapping function $\mathcal{L}^*$ can be calculated by the following equation after adopting asynchronous stochastic gradient descent (ASGD) [64] to optimize the loss function:

$$\begin{aligned} \mathcal{L}^* &= \arg \min_{\mathcal{L}, \psi} \mathcal{O}_{\text{ssl}}(\mathcal{L}, \psi, \mathcal{D}_t) \\ &= \sum_{r \in \mathcal{V}_t} \sum_{i=1}^k \mathbb{E}_{v^n \sim P_n(j)} \log(\sigma(-f'(\mathbb{X}^r, \mathbb{X}^{v^n}))) \\ &= \sum_{r \in \mathcal{V}_t} \sum_{i=1}^k y \cdot \log(\sigma(-\alpha_{r,v^n} (\mathbb{X}^r)^\top \mathbb{X}^{v^n})) \\ &\quad + (1 - y) \cdot \log(\sigma(-(\mathbb{X}^r)^\top \mathbb{X}^{v^n})). \end{aligned} \quad (12)$$

We summarize the training process of preserving the long-range dynamics by using the Graph SSL (LDPP-SSL) optimizer into Algorithm 1.

Algorithm 1 LDPP-SSL Training

Input: network snapshot G_t , pretrained mapping function $\mathcal{L}$.

Parameter: $\theta=5, lr=0.025$.

Output: enhanced temporal mapping function $\mathcal{L}^*$

```

1: change  $G_t$  into the directed graph  $G_t^D=(\mathcal{V}_t, \mathcal{E}_t^{\rightarrow}, \mathcal{A}_t^{\rightarrow})$ 
2: for  $r$  in  $\mathcal{V}_t$  do
3:    $(\mathbb{V}, \mathbb{E})=\text{DFS}(G_t^D, r)$ ;  $\mathbb{X} = \mathcal{L}(\mathbb{V})$ .
4:   for  $n$  in  $P_n(j \in \mathbb{V} \setminus r) \propto \lceil (d_j)^{3/4} \rceil$  do
5:     if  $n==r$  then
6:        $\alpha_{r,n}=0$ .
7:     else
8:        $\text{hop}(r, n) = \text{ShortestPathLength}(G_t^D, r, n)$ .
9:       if  $\text{hop}(r, n) > \theta$  then
10:         $\alpha_{r,n} = 1$ ;  $y=0$ .
11:      else
12:         $\alpha_{r,n} = \frac{1}{\text{hop}(r,n)}$ ;  $y=1$ .
13:      end if
14:    end if
15:     $\psi = \sigma(-(\alpha_{r,n} \cdot (\mathbb{X}^r)^\top \cdot \mathbb{X}^n))$ .
16:  end for
17:   $\mathcal{L}^* = \text{ASGD}(y, \psi, lr)$ .
18: end for
19: return  $\mathcal{L}^*$ .
```

C. Accumulated Forward Update

Because networks exhibit complex temporal properties in realistic scenarios, the topological structure will evolve constantly. Therefore, we integrate an AFU module into LongTNE to capture the global evolutionary regularities of temporal networks. As shown in Fig. 2(c), AFU leverages the seed vertex that co-occurs in multiple network snapshots to enhance the vertex embeddings of the network snapshot G_t . In comparison, other vertices are considered as *candidates* for obtaining the enhanced mapping function of the network snapshot G_{t+1} . Formally, the AFU paradigm can be represented as follows:

$$\begin{aligned} \mathbf{X}_t^E &\leftarrow: g(\mathcal{L}^*(\mathcal{A}_t, \mathbf{X}_t'), \mathbf{X}_{t-1}^E) \\ &= \mathcal{L}^*(\mathcal{A}_t, (\mathbf{X}_t \setminus \mathbf{X}_t(s)) \cup \mathbf{X}_{t-1}^E(s)) \cup \mathbf{X}_{t-1}^E(o) \end{aligned} \quad (13)$$

where $\mathbf{X}_t'$ is the initial embeddings of G_t , which is obtained by replacing the pretrained seed-vertex embeddings $\mathbf{X}_t(s)$ with the enhanced seed-vertex embeddings $\mathbf{X}_{t-1}^E(s)$ in G_{t-1} . s refers to the seed vertex. $\mathbf{X}_{t-1}^E(o)$ denotes the enhanced embedding of other vertex (o) in G_{t-1} . We summarize the AFU paradigm into Algorithm 2.

V. EXPERIMENTS

We evaluate LongTNE with three network mining tasks, that is, multihop link prediction (Mul-LP), network-temporal reconstruction (NTR), and temporal vertex recommendation (TVR). Experimental results could be used to answer the following questions.

- 1) *Deep Knowledge Discovery*: Can LongTNE accurately explore deep knowledge from temporal networks to support the Mul-LP task?

Algorithm 2 AFU Paradigm

Input: temporal network $G_T = \{G_1, \dots, G_t\}$, pretrained vertex embedding $\mathbf{X}_T = \{\mathbf{X}_1, \dots, \mathbf{X}_t\}$.

Output: Enhanced temporal network embedding $\mathbf{X}_t^E$

```

1:  $\mathbf{X}_1^E = \text{LDPP-SSL}(\mathbf{X}_1, G_1)$ .
2: for each  $G_i$  in  $\{G_2, G_3, \dots, G_t\}$  do
3:    $\mathbf{X}_i = (\mathbf{X}_i \setminus \mathbf{X}_i(s)) \cup (\mathbf{X}_{i-1}^E(s))$ .
4:    $\mathbf{X}_i^E = \text{LDPP-SSL}(\mathbf{X}_i, G_i)$ .
5:    $\mathbf{X}_i^E = \mathbf{X}_i^E \cup \mathbf{X}_{i-1}^E(o)$ .
6: end for
7:  $\mathbf{X}_t^E$  is the last round result of for loop.
8: return  $\mathbf{X}_t^E$ .
```

TABLE I
STATISTICS OF TEMPORAL NETWORKS

	$ \mathcal{V} $	$ \mathcal{E} $	$Avg.D$	$\#T$	$\#Snap$
RFA	10,608	191,107	29.652	-	2
MS	144,064	247,394	3.385	-	2
FOOD	153,439	225,265	2.455	-	3
ENRON	143	20,498	286.685	-	16
DBLP	28,085	236,894	16.870	27	4
TMALL	577,314	4,807,545	33.447	186	7

$Avg.D$ denotes the average degree of networks. $\#T$ denotes the number of timestamps. $\#Snap$ denotes the number of snapshots.

- 2) *Evolutionary Knowledge Discovery*: Can LongTNE effectively capture temporal information for learning the dynamic evolution of networks?
- 3) *Module Effectiveness*: How much do different modules contribute to the performance improvement of LongTNE?

A. Datasets

We use four discrete-time temporal networks [request for adminship (RFA),¹ movie summaries (MS) [65], FOOD,¹ and enron email communication network (Enron)²] and two continuous-time temporal networks (DBLP³ and TMALL⁴) to illustrate the effectiveness of LongTNE. The statistics of temporal networks are summarized in Table I. We also propose a time-step partition schema for two continuous-time temporal networks to divide their continuous timestamp into several discrete snapshots.

First, we fit the approximate distribution $\mathcal{F}(x)$ for temporal edges at different timestamps and determine its global temporal interval $[t_{\min}, t_{\max}]$. Thus, the enclosed area $\mathcal{S}$ under the distribution $\mathcal{F}(x)$ can be calculated by $\int_{t_{\min}}^{t_{\max}} \mathcal{F}(x) dx$. Afterward, we introduce a hyperparameter N that determines the number of network snapshots and assume that the divided area of each snapshot is equal, that is, $S_i = (\mathcal{S}/N)$. Therefore, for each time interval $[x_i, x_i + \delta]$, its divided area S_i can be estimated by $\int_{x_i}^{x_i+\delta} (\mathcal{F}(x_i) + \mathcal{F}(x_i + \delta)/2) dx_i$. In this way, we can sequentially obtain the division point δ by initializing $x_0 = t_{\min}$ and constraining $x_i + \delta < t_{\max}$.

For two continue-time temporal networks, we first obtain their edge distributions. As shown in Fig. 3, the distribution of

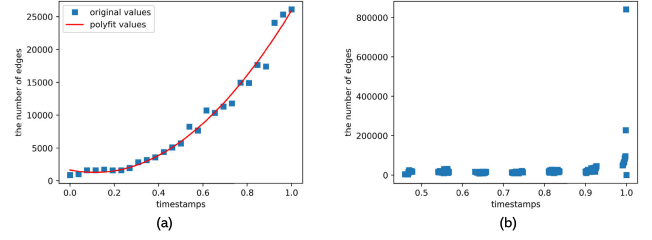


Fig. 3. Edge distribution of (a) DBLP and (b) TMALL.

TMALL is segmented, so we directly divide it into seven network snapshots according to the distribution peak. In contrast, the distribution of DBLP is smooth. Therefore, we first fit its distribution as $\mathcal{F}(x) = 3.126e4x^2 - 6.978e3x + 1641$ and then determine the temporal interval as $[0, 1]$. Afterward, we define the required number of network snapshots as $N = 4$ and divide the enclosed area under $\mathcal{F}(x)$ into equal areas. Finally, we sequentially estimate the δ value to obtain the division points and collect network edges into different snapshots. Here, δ is, respectively, estimated as 14, 6, 4, and 3.

B. Baselines

We compare LongTNE with the following static network embedding (NE) algorithms and TNE algorithms.

- 1) *Static NE Methods*: First, we compare with naive methods such as node2vec [17], DeepWalk [18], and large-scale information network embedding (LINE) [19]. Then, we employ role2vec [66], HOPE [54], network embeddings as matrix factorization (NetMF) [20], and Walklets [67] that access the HP of networks as baselines. Edges in different network snapshots are collected into one network to learn vertex embeddings. All these methods are conducted with the best parameters tuned with the Karateclub tool.⁵
- 2) *TNE Methods*: We employ two discrete-time TNE methods (DynamicTraid [23] and Deep embedding method for dynamic graph (DynGEM) [24]) and five continuous-time TNE methods (HTNE [44], temporal network embedding method with micro- and macro-dynamics (MMDNE) [45], global topology preserving dynamic network embedding (GloDyNE) [46], motif-preserving temporal network embedding (MTNE) [47], and DCTAN [29]). All these methods are carried out with the parameters released in their codes. Our networks are also processed to be consistent with the codes.

All experiments were conducted on K80 (GPU), 128-GB memory, or Tesla-V100 (GPU), 32-GB memory (for DCTAN).

C. Deep Knowledge Discovery

Deep (multihop) knowledge discovery on networks is an inference task of exploring long-distance associated vertices j for the given vertex r based on the network topology. Here, we employ Mul-LP to evaluate the performance of LongTNE in deep knowledge discovery.

Mul-LP aims to determine if there is an edge between two given vertices based on the absolute difference in positions between their corresponding embeddings. We sample 80% edges as the training data and take the other 20% edges as the

¹<http://snap.stanford.edu/data>

²<https://www.cs.cornell.edu/arb/data/email-Enron/>

³<https://github.com/VGraphRNN/VGRNN/tree/master/data>

⁴<https://tianchi.aliyun.com/dataset/>

⁵<https://github.com/benedekrozemberczki/karateclub>

TABLE II
EXPERIMENTAL RESULTS OF DIFFERENT METHODS ON THE MUL-LP TASK

METHODS	Macro- F_1							Area Under Curve (AUC)						
	RFA	FOOD	MS	ENRON	DBLP	TMALL	<i>Avg.</i>	RFA	FOOD	MS	ENRON	DBLP	TMALL	<i>Avg.</i>
DeepWalk	0.726	0.678	0.555	0.864	0.899	0.633	0.726	0.802	0.756	0.577	0.941	0.963	0.776	0.803
LINE	0.788	0.689	0.537	0.838	0.887	0.691	0.738	0.836	0.781	0.568	0.911	0.939	0.785	0.803
node2vec	0.722	0.789	0.570	0.855	0.890	0.782	0.768	0.792	0.875	0.623	0.914	0.958	0.860	0.837
role2vec	0.666	0.737	0.730	0.801	0.859	OOM	0.759	0.725	0.823	0.796	0.868	0.926	OOM	0.828
HOPE	0.850	0.409	0.720	0.919	0.765	0.713	0.729	0.922	0.657	0.751	0.974	0.861	0.722	0.815
Walklets	0.811	0.660	0.693	0.836	0.893	0.775	0.778	0.880	0.710	0.762	0.919	0.952	0.783	0.834
NetMF	0.843	0.734	0.764	0.878	0.858	0.790	0.811	0.919	0.815	0.829	0.952	0.933	0.838	0.881
DynGEM	0.728	0.699	0.701	0.869	0.847	0.781	0.771	0.832	0.768	0.796	0.919	0.866	0.836	0.836
DynTriad	0.741	0.722	0.729	0.877	0.885	0.803	0.793	0.844	0.805	0.800	0.923	0.906	0.855	0.856
HTNE	0.770	0.786	0.722	0.910	0.917	0.822	0.821	0.848	0.869	0.792	0.966	0.939	0.905	0.887
MMDNE	0.803	0.785	0.754	0.875	0.911	0.803	0.822	0.884	0.866	0.825	0.950	0.936	0.888	0.892
MTNE	0.867	0.767	0.664	0.883	0.935	0.900	0.836	0.929	0.845	0.724	0.949	0.970	0.962	0.898
GloDyNE	0.726	0.787	0.528	0.876	0.876	OOM	0.759	0.799	0.872	0.570	0.938	0.945	OOM	0.825
DCTAN	0.671	OOM	OOM	0.557	0.607	OOM	0.612	0.731	OOM	OOM	0.606	0.640	OOM	0.659
LongTNE(node2vec)*	0.782	0.864	0.672	0.859	0.892	0.911	0.830	0.863	0.931	0.733	0.928	0.942	0.957	0.892
LongTNE(HOPE)*	0.819	0.711	0.760	0.901	0.769	0.715	0.779	0.904	0.790	0.813	0.965	0.863	0.726	0.844
LongTNE(NetMF)*	0.862	0.747	0.788	0.880	0.859	0.866	0.817	0.935	0.834	0.861	0.954	0.934	0.882	0.900
LongTNE(HTNE)*	0.771	0.797	0.731	0.911	0.927	0.870	0.835	0.853	0.875	0.796	0.967	0.958	0.932	0.897
LongTNE(MMDNE)*	0.813	0.795	0.763	0.878	0.922	0.893	0.844	0.891	0.871	0.832	0.949	0.946	0.956	0.908
LongTNE(MTNE)*	0.871	0.774	0.675	0.883	0.945	0.925	0.846	0.938	0.847	0.733	0.950	0.987	0.968	0.904
LongTNE(GloDyNE)*	0.781	0.864	0.633	0.881	0.877	NULL	0.807	0.863	0.930	0.687	0.947	0.944	NULL	0.874
GAIN (%)	+1.64	+7.07	+4.29	+0.17	+0.56	+3.86	+2.93	+2.19	+3.99	+4.87	+0.24	+0.34	+3.51	+2.52

* denotes p -value < 0.05 , indicating that the improvements are significant. OOM denotes the “Out-Of-Memory” and NULL indicates the null value as the experimental condition is missing. The best result in each network is boldfaced. The best baseline is italicized and underlined.

test data. Meanwhile, we ensure the test edges are generated from the training data by at least two hops. Based on vertex embeddings, we perform fivefold experiments on test data with different negative edges. In each evaluation, the inner product is used to compute similarities between two given vertices and predict the probability of link establishments.

The averaged Macro- F_1 and AUC are reported in Table II. It can be seen that the following holds.

- 1) Applying LongTNE to different NE methods achieves improvement in all temporal networks. Overall, LongTNE improves Macro- F_1 by 2.93% and AUC by 2.52% on average. In the best situation, it obtains a 7.07% increase in Macro- F_1 on FOOD and a 4.87% increase in AUC on MS.
- 2) LongTNE is more useful for mining deep knowledge from sparse temporal networks. LongTNE achieves about 5.68% gains in Macro- F_1 and 4.43% gains in AUC on FOOD and MS, which is about twice that of dense temporal networks.

The remarkable improvements are attributed to LongTNE’s exploration of the high-order nonlinear structure of temporal networks. In LongTNE, the long-range dynamics of vertices are converted into a tree structure by constructing the hierarchical context, which facilitates the automatic construction of contrastive views for two long-path reachable vertices, thereby establishing pseudo-supervision signals for unlabeled graphs. What is more, LongTNE employs the Graph SSL strategy to optimize model parameters based on the augmented graph data, which preserves the original network structure and mines the potential topological associations. Also, LongTNE has learned adaptive weights within positive samples to simulate the variable proximity of vertex pairs with different hops.

It demonstrates that LongTNE refines the learning granularity of nonlinear high-order information, which contributes to performance improvements.

To further elucidate the effectiveness of LongTNE, we cluster the vertex pairs with the same hops in MS and TMALL. We then use four baselines and LongTNE to perform Mul-LP on different datasets. The Macro- F_1 and AUC values are shown in Fig. 4, from which we observe that LongTNE obtains higher performance between two long-path reachable vertices. Therefore, we can conclude that LongTNE has a solid ability to discover multihop knowledge from temporal networks as it explores the long-range proximity of vertices.

D. Evolutionary Knowledge Discovery

Evolutionary knowledge discovery on networks is to capture distinct dynamic features of network snapshots at different timestamps. Therefore, we adopt the NTR and TVR tasks to evaluate the effectiveness of LongTNE on evolutionary knowledge discovery.

NTR aims to reconstruct edges in different network snapshots based on the proximity between two vertex embeddings. We first sample 10% edges from temporal networks as the evaluation data. Then, we train a logistic regression (LR) classifier to discriminate links at different timestamps and compute the averaged Macro- F_1 and AUC in fivefold experiments. Here, we compare LongTNE with TNE methods for the sake of fairness. The results are shown in Table III.

The results indicate that LongTNE(MMDNE) outperforms other methods in most cases, which can be traced back to as follows.

- 1) MMDNE employs distribution constraint and linking rate to learn the global evolution of temporal networks,

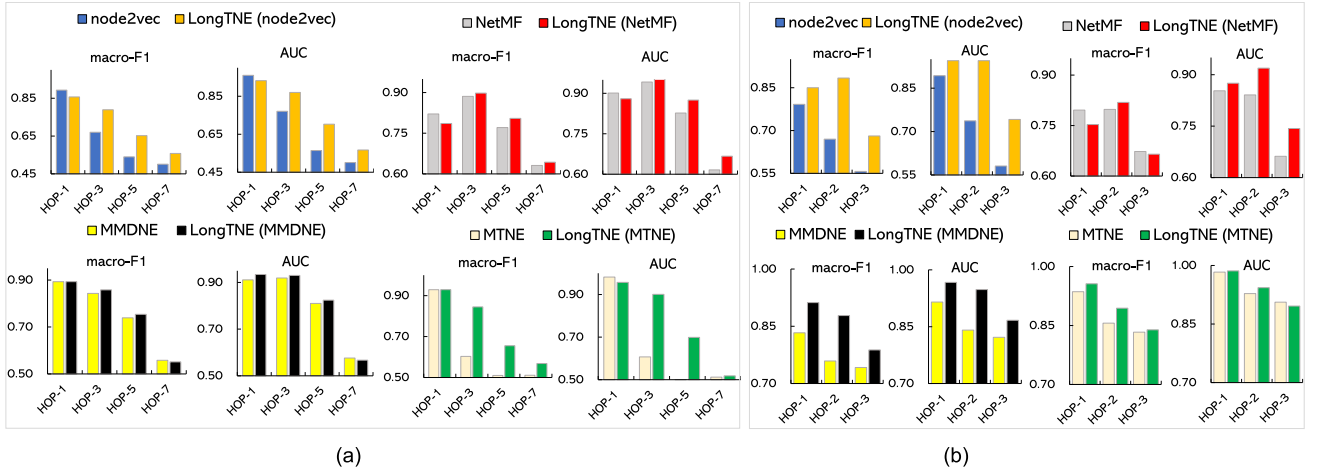


Fig. 4. Experimental results (Macro- F_1 and AUC) of the Mul-LP task on vertex pairs with different hops. (a) MS network. (b) TMALL network.

TABLE III
EXPERIMENTAL RESULTS OF TNE METHODS ON THE NTR TASK

METHODS	Macro- F_1						Area Under Curve (AUC)					
	RFA	FOOD	MS	ENRON	DBLP	TMALL	RFA	FOOD	MS	ENRON	DBLP	TMALL
DynTriad	0.729	0.660	0.674	0.049	0.362	0.083	0.811	0.931	0.566	0.622	0.833	0.548
HTNE	0.868	0.663	0.714	0.058	0.379	0.145	0.913	0.945	0.612	0.653	0.836	0.642
MMDNE	0.869	0.663	0.714	0.056	0.406	0.178	0.919	0.943	0.628	0.639	0.838	0.669
MTNE	0.793	0.663	0.717	0.077	0.391	0.074	0.831	0.925	0.545	0.635	0.838	0.504
GloDyNE	0.697	0.664	0.696	0.072	0.375	OOM	0.641	0.957	0.607	0.629	0.836	OOM
DCTAN	0.876	OOM	OOM	0.076	0.442	OOM	0.931	OOM	OOM	0.635	0.843	OOM
LongTNE(HTNE)*	0.870	0.664	0.718	0.063	0.379	0.147	0.924	0.956	0.629	0.663	0.840	0.648
LongTNE(MMDNE)*	0.889	0.664	0.736	0.077	0.413	0.182	0.927	0.961	0.734	0.642	0.846	0.674
LongTNE(MTNE)*	0.861	0.664	0.730	0.086	0.392	0.099	0.893	0.948	0.615	0.668	0.841	0.544
LongTNE(GloDyNE)*	0.812	0.665	0.700	0.082	0.381	NULL	0.784	0.971	0.663	0.657	0.844	NULL

* denotes p -value < 0.05 , indicating that the improvements are significant. OOM denotes the “Out-Of-Memory” and NULL indicates the null value as the experimental condition is missing. The best result in each network is boldfaced.

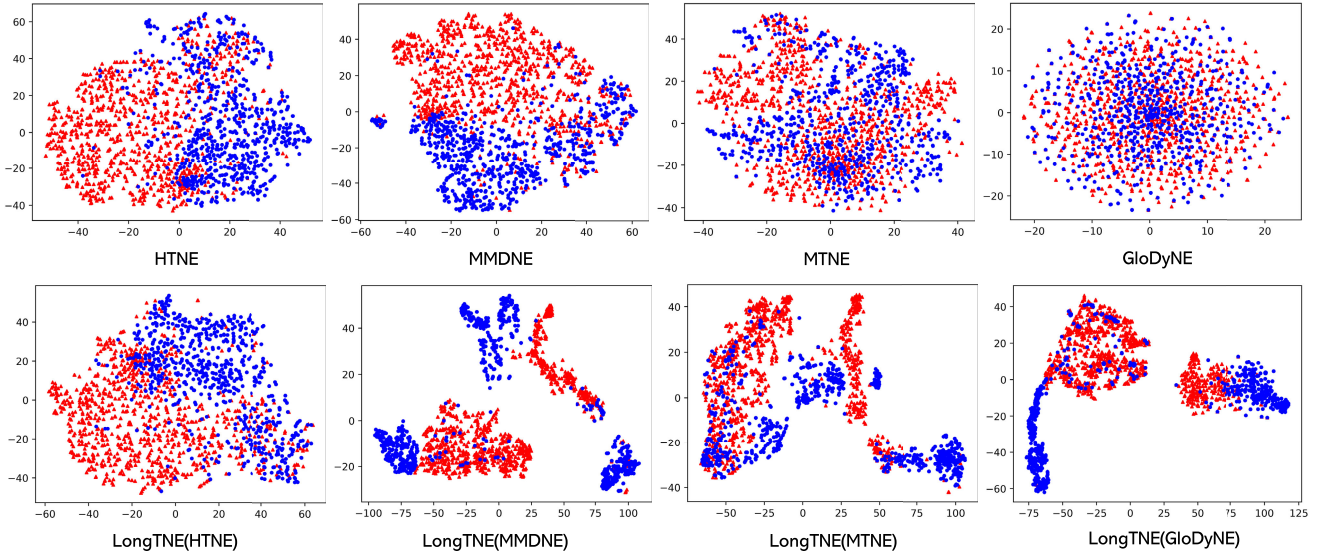


Fig. 5. Visualization of vertex embeddings on the NTR task, where the temporal network is selected to RFA. The blue circles represent edges in the network snapshot G_1 , while the red triangles represent edges in the network snapshot G_2 .

which provides LongTNE with high-quality pretrained vertex embeddings.

- LongTNE explores deep-seated topological features via the LDPP module and captures the deep associations among vertices. Therefore, LongTNE can continue to boost the performance of MMDNE on the NTR task.

- LongTNE preserves prior knowledge learned from the latest historical networks by integrating the AFU module, thereby distinguishing different temporal attributes of an edge.

To intuitively show the performance of LongTNE in NTR, Fig. 5 displays the visualization of vertex embeddings learned

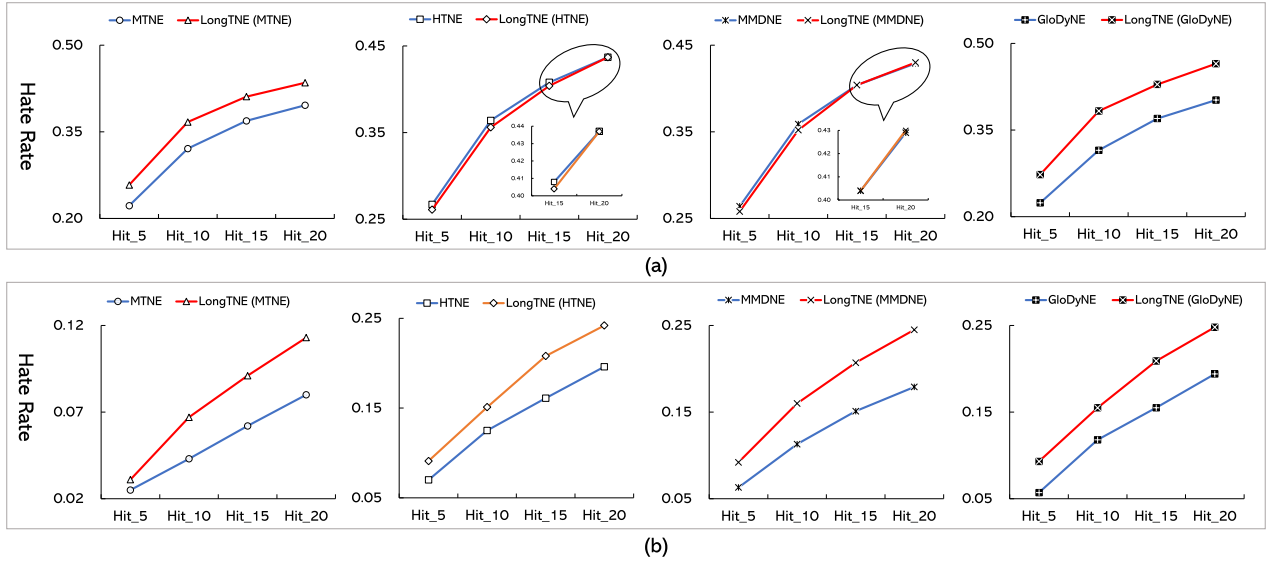


Fig. 6. Hit_k evaluations of different TNE methods on the TVR task. The networks are selected as (a) FOOD and (b) MS.

by different baselines and LongTNE. Each point is a reconstructed edge in different network snapshots. Specifically, we acquire edge vectors of each (i, j) by calculating $|\vec{u}_i - \vec{u}_j|$. Then, we reduce edge vectors to two dimensions through the t-distributed stochastic neighbor embedding (TSNE)⁶ tool. Finally, we plot 2-D images of the classification results using matplotlib.⁷ After applying LongTNE, we observe that edges that appeared at different timestamps have a more apparent boundary, which shows that LongTNE has a better ability to discriminate dynamic links of temporal networks.

TVR aims to predict top- k possible neighbors at time t for each v_i in the network snapshot before time t . First, we construct a dataset by sampling 1000 source vertices and collecting 2000 edges containing the source vertices. Afterward, we calculate the Euclidean distance for all possible vertex pairs and derive the top- k vertices with the highest score as candidates. Finally, we calculate the *Hit Rate* of different methods to evaluate their performance on this task. The results shown in Fig. 6 elucidate that LongTNE improves the performance of all baselines under the Hit_k ($k = \{5, 10, 15, 20\}$) metrics. Among all LongTNE settings, LongTNE(GloDyNE), respectively, reaches the Hit₂₀ of 46.5% and 24.8% on the FOOD network and the MS network. The excellent performance proves the effectiveness of LongTNE in capturing evolutionary features from temporal networks.

E. Module Effectiveness

1) *Long-Distance Dynamic Proximity Preservation*: As mentioned before, LDPP serves to learn the HP of networks. Similarly, the recent graph transformer performs well in capturing deep topological features. Therefore, we apply the LDPP module and the structural self-attention (SSA) [48] to static NE methods and observe the performance of vertex embeddings under different settings. The Macro- F_1 of Mul-LP is summarized in Fig. 7.

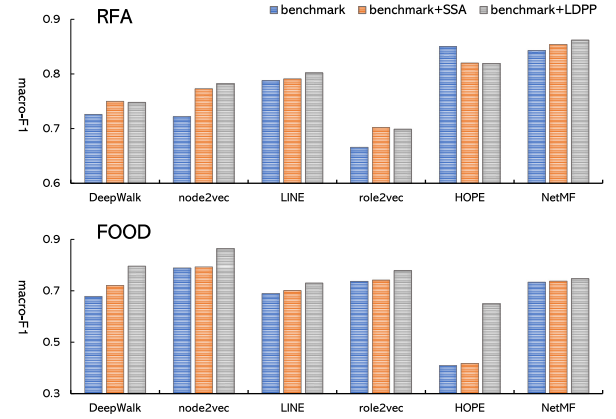


Fig. 7. Results of using different mechanisms to enhance vertex embeddings.

Results show that both LDPP and SSA can improve Macro- F_1 for most baselines but LDPP performs better than SSA on the sparse network (i.e., FOOD). This is because LDPP establishes adaptive dynamic weights for vertices in a long path, while SSA aggregates the neighborhood information of vertices step by step by using attention weights.

2) *Graph Self-Supervised Learning Optimizer*: In LDPP, we employ Graph SSL as the optimizer to preserve the long-range dynamic proximity among vertices. To illustrate its effectiveness, we compare it with four strategies.

- 1) *Un-Supervised (UN.)*: We use graph autoencoder (GAE) to reconstruct the adjacent matrix of different network snapshots. we do not use α and θ here.
- 2) *Semi-Supervised (SEMI.)*: We annotate all vertices in the long-distance traversal tree as positive samples and do not use parameters α and θ to supplement other information.
- 3) *Weak-Supervised (WEAK.)*: We use the parameter θ to build contrastive views for vertices but do not employ the parameter α to refine the learning granularity of long-range dynamics.

⁶scikit-learn.org/stable/modules/generated/sklearn.manifold.TSNE.html

⁷https://matplotlib.org/3.1.1/api/pyplot_summary.html

TABLE IV

RESULTS OF DIFFERENT OPTIMIZATION STRATEGIES ON MUL-LP

Macro- F_1	RFA	FOOD	DBLP	TMALL
UN.	0.8483	0.6817	0.8871	0.8638
SEMI.	0.7768	0.6327	0.7918	0.8007
WEAK.	0.8705	0.7729	0.9450	0.9238
SELF.	0.8714	0.7741	0.9453	0.9253
w/o SELF.	0.8789	0.7703	0.9380	0.9244
AUC	RFA	FOOD	DBLP	TMALL
UN.	0.9225	0.7459	0.9437	0.9422
SEMI.	0.8721	0.7185	0.8922	0.9100
WEAK.	0.9311	0.8440	0.9819	0.9641
SELF.	0.9378	0.8474	0.9872	0.9682
w/o SELF.	0.9213	0.8471	0.9810	0.9602

The best result in each network is boldfaced.

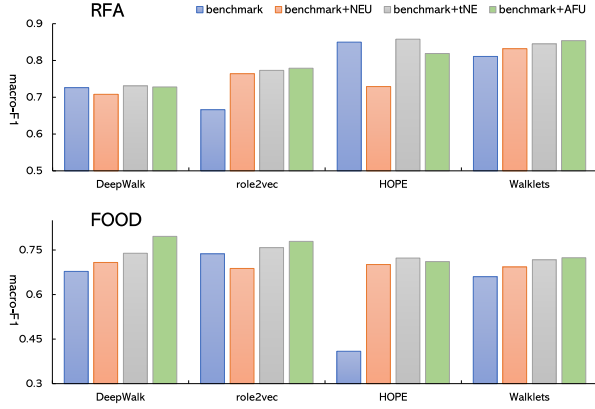


Fig. 8. Experimental results of NEU, tNE, and AFU on the Mul-LP task. The networks here are RFA and FOOD.

- 4) *Without Self-Supervised (w/o SELF)*: We use neighbors $\mathcal{N}_i$ of the vertex i in each network snapshot to update vertex embeddings. We do not configure α and θ here.

Results of LongTNE(MTNE) with different settings on Mul-LP are summarized in Table IV, which proves that Graph SSL is indeed useful for integrating long-range dynamics to enhance vertex embeddings. Moreover, we find that without using the pseudo-label signals to distinguish topological roles of vertices in the long-distance traversal tree, the performance would be even worse than the naive unsupervised method.

3) *Accumulated Forward Update*: We compare AFU with network embedding update (NEU) [52] and tNodeEmb (tNE) [68] to demonstrate their effectiveness in capturing global temporal features. NEU uses matrix factorization to learn the global features of different network snapshots while tNE aligns the initialized static vertex embedding of a vertex at different timestamps. Here, we utilize the mode of “benchmark + {NEU, tNE, or AFU}” to perform Mul-LP on RFA and FOOD. The Macro- F_1 in Fig. 8 illustrates that AFU improves the performance of all baselines and is more stable than NEU in most cases. Thus, with the fact that both AFU, tNE, and NEU can capture HP, LongTNE equipped with the AFU module is more robust.

VI. DISCUSSION

A. Selection of Parameter θ

LongTNE involves a hyperparameter $\theta = 5$ in LDPP to prepare training samples. Here, we provide the evidence for

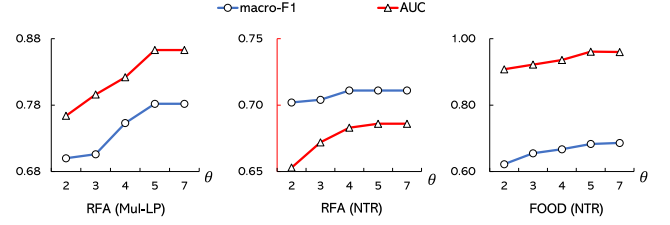


Fig. 9. Results of LongTNE on Mul-LP and NTR under different θ .

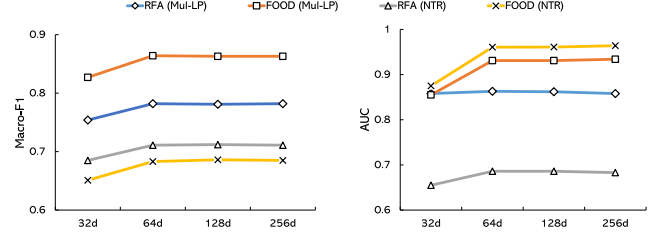


Fig. 10. Results of LongTNE on Mul-LP and NTR under different d .

TABLE V

WALL-CLOCK TIME (IN SECONDS) TO OBTAIN VERTEX EMBEDDINGS

METHODS	RFA	FOOD	MS	ENRON	DBLP	TMALL
MTNE	1,613	2,522	2,713	308	2,455	101,942
LongTNE	228	150	162	7.2	1,596	5,904
w/ WEAK.	227	152	164	7.0	1,653	5,933
w/ SEMI.	311	249	257	6.3	608	9,107
w/o SELF.	32	225	233	3.8	23	414

the setting based on the experimental results in Fig. 9. We use LongTNE(node2vec) to perform the Mul-LP and NTR tasks under $\theta \in \{2, 3, 4, 5, 7\}$ and find that the results are improved in all situations from $\theta = 3$ to $\theta = 5$ and are stable when $\theta \geq 5$.

B. Selection of Dimension Size d

Based on different dimension sizes $d = \{32, 64, 128, 256\}$, we also conduct Mul-LP and NTR on RFA and FOOD, where the parameter θ is fixed with 5. The experimental results in Fig. 10 say that when $d = 64$, the performance of LongTNE(node2vec) is the best.

C. Training Efficiency

In Algorithm 1, LongTNE is an enhancement framework that aims to fine-tune the pretrained vertex embedding rather than encoding networks from scratch. Therefore, the time complexity of LongTNE is dependent on the optimization process and can be computed as $\mathcal{O}(ndT|\mathcal{V}||\mathcal{E}|)$, where n is the number of sampling vertices, d is the size of dimensions, and T is the number of network snapshots. $|\mathcal{V}|$ and $|\mathcal{E}|$ denote the number of vertices and edges in each G^D , respectively. We train LongTNE with different strategies five times on different networks (only one time on TMALL) based on vertex embeddings obtained by MTNE, and their average training time is demonstrated in Table V.

VII. CONCLUSION

In this study, we propose LongTNE that integrates Graph SSL to explore long-range dynamics for TNE. The effectiveness and extension of LongTNE are verified through

multiple network mining tasks on six temporal networks. However, LongTNE is currently tailored for knowledge discovery in relational networks with temporal attributes. In the future, we will combine LongTNE with large language models (LLMs) to unify multiview representations of spatial sequence data and temporal topological data, thus empowering LongTNE with tackling anomaly detection or action recognition tasks.

REFERENCES

- [1] L. Shao, D. Wu, and X. Li, "Learning deep and wide: A spectral method for learning deep networks," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 25, no. 12, pp. 2303–2308, Dec. 2014.
- [2] S. Wang, L. He, B. Cao, C.-T. Lu, P. S. Yu, and A. B. Ragin, "Structural deep brain network mining," in *Proc. 23rd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, Aug. 2017, pp. 475–484.
- [3] H. Hong, X. Li, and M. Wang, "GANE: A generative adversarial network embedding," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 31, no. 7, pp. 2325–2335, Jul. 2020.
- [4] L. Cai and S. Ji, "A multi-scale approach for graph link prediction," in *Proc. AAAI Conf. Artif. Intell.*, vol. 34, pp. 3308–3315, 2020.
- [5] X. Wang et al., "Heterogeneous graph attention network," in *Proc. World Wide Web Conf.*, 2019, pp. 2022–2032.
- [6] X. Shen, Q. Dai, S. Mao, F.-L. Chung, and K.-S. Choi, "Network together: Node classification via cross-network deep network embedding," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 32, no. 5, pp. 1935–1948, May 2021.
- [7] P. Duan, C. Zhou, and Y. Liu, "Dynamic graph representation learning via coupling-process model," *IEEE Trans. Neural Netw. Learn. Syst.*, early access, pp. 1–13, Mar. 24, 2023, doi: [10.1109/TNNLS.2023.3257488](https://doi.org/10.1109/TNNLS.2023.3257488).
- [8] H.-H. Shuai et al., "A comprehensive study on social network mental disorders detection via online social media mining," *IEEE Trans. Knowl. Data Eng.*, vol. 30, no. 7, pp. 1212–1225, Jul. 2018.
- [9] C. Shi, B. Hu, W. X. Zhao, and P. S. Yu, "Heterogeneous information network embedding for recommendation," *IEEE Trans. Knowl. Data Eng.*, vol. 31, no. 2, pp. 357–370, Feb. 2018.
- [10] L. Liu, J. Chen, H. Wu, J. Zhen, G. Li, and L. Lin, "Physical-virtual collaboration modeling for intra- and inter-station metro ridership prediction," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 4, pp. 3377–3391, Apr. 2022.
- [11] Y. Zhu et al., "Hybrid-order representation learning for electricity theft detection," *IEEE Trans. Ind. Informat.*, vol. 19, no. 2, pp. 1248–1259, Feb. 2023.
- [12] T. Chen, T. Pu, H. Wu, Y. Xie, L. Liu, and L. Lin, "Cross-domain facial expression recognition: A unified evaluation benchmark and adversarial graph learning," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 12, pp. 9887–9903, Dec. 2022.
- [13] Y. Liu, K. Wang, G. Li, and L. Lin, "Semantics-aware adaptive knowledge distillation for sensor-to-vision action recognition," *IEEE Trans. Image Process.*, vol. 30, pp. 5573–5588, 2021.
- [14] H. Cai, V. W. Zheng, and K. C. Chang, "A comprehensive survey of graph embedding: Problems, techniques, and applications," *IEEE Trans. Knowl. Data Eng.*, vol. 30, no. 9, pp. 1616–1637, Sep. 2018.
- [15] P. Cui, X. Wang, J. Pei, and W. Zhu, "A survey on network embedding," *IEEE Trans. Knowl. Data Eng.*, vol. 31, no. 5, pp. 833–852, May 2018.
- [16] H. Chen, B. Perozzi, Y. Hu, and S. Skiena, "Harp: Hierarchical representation learning for networks," in *Proc. AAAI Conf. Artif. Intell.*, 2018, vol. 32, no. 1, pp. 1–12.
- [17] A. Grover and J. Leskovec, "Node2vec: Scalable feature learning for networks," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, Aug. 2016, pp. 855–864.
- [18] B. Perozzi, R. Al-Rfou, and S. Skiena, "DeepWalk: Online learning of social representations," in *Proc. 20th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, Aug. 2014, pp. 701–710.
- [19] J. Tang, M. Qu, M. Wang, M. Zhang, J. Yan, and Q. Mei, "LINE: Large-scale information network embedding," in *Proc. 24th Int. Conf. World Wide Web*, May 2015, pp. 1067–1077.
- [20] J. Qiu, Y. Dong, H. Ma, J. Li, K. Wang, and J. Tang, "Network embedding as matrix factorization: Unifying deepwalk, LINE, PTE, and node2vec," in *Proc. 11th ACM Int. Conf. Web Search Data Mining*, Feb. 2018, pp. 459–467.
- [21] A. Li, S. P. Cornelius, Y.-Y. Liu, L. Wang, and A.-L. Barabási, "The fundamental advantages of temporal networks," *Science*, vol. 358, no. 6366, pp. 1042–1046, Nov. 2017.
- [22] L. Du, Y. Wang, G. Song, Z. Lu, and J. Wang, "Dynamic network embedding : An extended approach for skip-gram based network embedding," in *Proc. 27th Int. Joint Conf. Artif. Intell.*, Jul. 2018, pp. 2086–2092.
- [23] L. Zhou, Y. Yang, X. Ren, F. Wu, and Y. Zhuang, "Dynamic network embedding by modeling triadic closure process," in *Proc. AAAI Conf. Artif. Intell.*, 2018, vol. 32, no. 1, pp. 1–14.
- [24] N. Kamra, P. Goyal, X. He, and Y. Liu, "Dyngem: Deep embedding method for dynamic graphs," in *Proc. IJCAI Int. Workshop Represent. Learn. Graphs (ReLiG)*, 2017, pp. 1–20.
- [25] D. Zhu, P. Cui, Z. Zhang, J. Pei, and W. Zhu, "High-order proximity preserved embedding for dynamic networks," *IEEE Trans. Knowl. Data Eng.*, vol. 30, no. 11, pp. 2134–2144, Nov. 2018.
- [26] G. H. Nguyen, J. B. Lee, R. A. Rossi, N. K. Ahmed, E. Koh, and S. Kim, "Continuous-time dynamic network embeddings," in *Proc. Companion Web Conf. Web Conf.*, 2018, pp. 969–976.
- [27] S. Kumar, X. Zhang, and J. Leskovec, "Predicting dynamic embedding trajectory in temporal interaction networks," in *Proc. 25th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, Jul. 2019, pp. 1269–1278.
- [28] X. Chang et al., "Continuous-time dynamic graph learning via neural interaction processes," in *Proc. 29th ACM Int. Conf. Inform. Knowl. Manag.*, 2020, pp. 145–154.
- [29] S. Tang, Z. Meng, and S. Liang, "Dynamic co-embedding model for temporal attributed networks," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 3, pp. 1–15, Mar. 2024.
- [30] T. Zheng et al., "Temporal aggregation and propagation graph neural networks for dynamic representation," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 10, pp. 1–17, Oct. 2023.
- [31] M. Farajtabar, Y. Wang, M. Gomez-Rodriguez, S. Li, H. Zha, and L. Song, "COEVOLVE: A joint point process model for information diffusion and network co-evolution," in *Proc. Annu. Conf. Neural Inf. Process. Syst.*, 2015, pp. 1954–1962.
- [32] C. Ai-Xiang, F. Yan, S. Ming-Sheng, C. Duan-Bing, and Z. Tao, "Emergence of local structures in complex network: Common neighborhood drives the network evolution," *Acta Phys. Sinica*, vol. 60, no. 3, pp. 803–808, 2011.
- [33] R. K. Nath, W. N. T. Fafous, K. C. Das, and Y. Shang, "Common neighborhood energy of commuting graphs of finite groups," *Symmetry*, vol. 13, no. 9, p. 1651, Sep. 2021.
- [34] H. Tian and R. Zafarani, "Exploiting common neighbor graph for link prediction," in *Proc. 29th ACM Int. Conf. Inf. Knowl. Manag.*, Oct. 2020, pp. 3333–3336.
- [35] A. Taheri, K. Gimpel, and T. Berger-Wolf, "Learning to represent the evolution of dynamic graphs with recurrent models," in *Proc. Companion World Wide Web Conf.*, May 2019, pp. 301–307.
- [36] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, and S. Y. Philip, "A comprehensive survey on graph neural networks," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 32, no. 1, pp. 4–24, Mar. 2020.
- [37] D. Q. Nguyen, T. D. Nguyen, and D. Phung, "A self-attention network based node embedding model," in *Proc. Joint Eur. Conf. Mach. Learn. Knowl. Discovery Databases*, 2020, pp. 364–377.
- [38] M. Xu, H. Wang, B. Ni, H. Guo, and J. Tang, "Self-supervised graph-level representation learning with local and global structure," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 11548–11558.
- [39] L. Yang, L. Zhang, and W. Yang, "Graph adversarial self-supervised learning," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 34, 2021, pp. 1–17.
- [40] X. Wang, N. Liu, H. Han, and C. Shi, "Self-supervised heterogeneous graph neural network with co-contrastive learning," in *Proc. 27th ACM SIGKDD Conf. Knowl. Discovery Data Mining*, Aug. 2021, pp. 1726–1736.
- [41] Y. Zheng, M. Jin, Y. Liu, L. Chi, K. T. Phan, and Y. P. Chen, "Generative and contrastive self-supervised learning for graph anomaly detection," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 12, Dec. 2023.
- [42] W. Zhang, L. Yang, S. Geng, and S. Hong, "Self-supervised time series representation learning via cross reconstruction transformer," *IEEE Trans. Neural Netw. Learn. Syst.*, early access, pp. 1–10, Jul. 21, 2023, doi: [10.1109/TNNLS.2023.3292066](https://doi.org/10.1109/TNNLS.2023.3292066).
- [43] R. Trivedi, M. Farajtabar, P. Biswal, and H. Zha, "DyRep: Learning representations over dynamic graphs," in *Proc. 7th Int. Conf. Learn. Represent. (ICLR)*, New Orleans, LA, USA: OpenReview.net, May 2019. [Online]. Available: <https://openreview.net/forum?id=HyePrhR5KX>

- [44] Y. Zuo, G. Liu, H. Lin, J. Guo, X. Hu, and J. Wu, "Embedding temporal network via neighborhood formation," in *Proc. 24th ACM SIGKDD Int. Conf. Knowl. Discov. Data Min.*, 2018, pp. 2857–2866.
- [45] Y. Lu, X. Wang, C. Shi, P. S. Yu, and Y. Ye, "Temporal network embedding with micro-and macro-dynamics," in *Proc. 28th ACM Int. Conf. Inf. Knowl. Manag.*, 2019, pp. 469–478.
- [46] C. Hou, H. Zhang, S. He, and K. Tang, "GloDyNE: Global topology preserving dynamic network embedding," *IEEE Trans. Knowl. Data Eng.*, vol. 34, no. 10, pp. 4826–4837, Oct. 2022.
- [47] H. Huang, Z. Fang, X. Wang, Y. Miao, and H. Jin, "Motif-preserving temporal network embedding," in *Proc. 29th Int. Joint Conf. Artif. Intell.*, Jul. 2020, pp. 1237–1243.
- [48] A. Sankar, Y. Wu, L. Gou, W. Zhang, and H. Yang, "DySAT: Deep neural representation learning on dynamic graphs via self-attention networks," in *Proc. 13th Int. Conf. Web Search Data Mining*, Jan. 2020, pp. 519–527.
- [49] A. Pareja et al., "EvolveGCN: Evolving graph convolutional networks for dynamic graphs," in *Proc. AAAI Conf. Artif. Intell.*, Apr. 2020, vol. 34, no. 4, pp. 5363–5370.
- [50] M. Yang, M. Zhou, M. Kalandar, Z. Huang, and I. King, "Discrete-time temporal network embedding via implicit hierarchical learning in hyperbolic space," in *Proc. 27th ACM SIGKDD Conf. Knowl. Discovery Data Mining*, Aug. 2021, pp. 1975–1985.
- [51] P. Jiao et al., "Temporal network embedding for link prediction via VAE joint attention mechanism," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 33, no. 12, pp. 7400–7413, Dec. 2022.
- [52] C. Yang, M. Sun, Z. Liu, and C. Tu, "Fast network embedding enhancement via high order proximity approximation," in *Proc. 26th Int. Joint Conf. Artif. Intell.*, Aug. 2017, pp. 3894–3900.
- [53] S. Cao, W. Lu, and Q. Xu, "GraRep: Learning graph representations with global structural information," in *Proc. 24th ACM Int. Conf. Inf. Knowl. Manage.*, Oct. 2015, pp. 891–900.
- [54] M. Ou, P. Cui, J. Pei, Z. Zhang, and W. Zhu, "Asymmetric transitivity preserving graph embedding," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discov. Data Min.*, 2016, pp. 1105–1114.
- [55] Z. Qiu, W. Hu, J. Wu, W. Liu, B. Du, and X. Jia, "Temporal network embedding with high-order nonlinear information," in *Proc. AAAI Conf. Artif. Intell.*, 2020, vol. 34, no. 4, pp. 5436–5443.
- [56] Y. Liu et al., "Graph self-supervised learning: A survey," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 6, pp. 5879–5900, Jun. 2023.
- [57] J. Wu et al., "Self-supervised graph learning for recommendation," in *Proc. 44th Int. ACM SIGIR Conf. Res. Develop. Inf. Retr.*, Jul. 2021, pp. 726–735.
- [58] Y. Liu, K. Wang, L. Liu, H. Lan, and L. Lin, "TCGL: Temporal contrastive graph for self-supervised video representation learning," *IEEE Trans. Image Process.*, vol. 31, pp. 1978–1993, 2022.
- [59] X. Yang, C. Deng, Z. Dang, K. Wei, and J. Yan, "Self-SAGCN: Self-supervised semantic alignment for graph convolution network," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2021, pp. 16775–16784.
- [60] J. Qiu et al., "GCC: Graph contrastive coding for graph neural network pre-training," in *Proc. 26th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, 2020, pp. 1150–1160.
- [61] Y. You, T. Chen, Y. Sui, T. Chen, Z. Wang, and Y. Shen, "Graph contrastive learning with augmentations," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2020, pp. 5812–5823.
- [62] A. Barrat and M. Weigt, "On the properties of small-world network models," *Eur. Phys. J. B*, vol. 13, no. 3, pp. 547–560, Jan. 2000.
- [63] C.-I. Hsu and H.-H. Shih, "Small-world network theory in the study of network connectivity and efficiency of complementary international airline alliances," *J. Air Transp. Manage.*, vol. 14, no. 3, pp. 123–129, May 2008.
- [64] B. Recht, C. Re, S. Wright, and F. Niu, "Hogwild! A lock-free approach to parallelizing stochastic gradient descent," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 24, 2011, pp. 693–701.
- [65] D. Bamman, B. O'Connor, and N. A. Smith, "Learning latent personas of film characters," in *Proc. 51st Annu. Meeting Assoc. Comput. Linguistics*, 2013, pp. 352–361.
- [66] N. K. Ahmed et al., "Learning role-based graph embeddings," 2018, *arXiv:1802.02896*.
- [67] B. Perozzi, V. Kulkarni, H. Chen, and S. Skiena, "Don't walk, skip!: Online learning of multi-scale network embeddings," in *Proc. IEEE/ACM Int. Conf. Adv. Social Netw. Anal. Mining*, Jul. 2017, pp. 258–265.

- [68] U. Singer, I. Guy, and K. Radinsky, "Node embedding over temporal graphs," in *Proc. 28th Int. Joint Conf. Artif. Intell. (IJCAI)*, Aug. 2019, pp. 4605–4612.



Zhizheng Wang received the M.S. and Ph.D. degrees from Dalian University of Technology, Dalian, China, in 2018 and 2023, respectively.

He is currently a Post-Doctoral Fellow at the National Institutes of Health (NIH), Bethesda, MD, USA. His research interests include data mining, representation learning, and large language models.



Yuanyuan Sun (Member, IEEE) received the B.E., M.S., and Ph.D. degrees from Dalian University of Technology, Dalian, China, in 2000, 2003, and 2009, respectively.

She is currently a Professor with the College of Computer Science, Dalian University of Technology. Her research interests include natural language processing, nonlinear theory and applications, and machine learning.



Zhihao Yang (Senior Member, IEEE) received the Ph.D. degree in computer science from Dalian University of Technology, Dalian, China, in 2008.

He is currently a Professor with the School of Computer Science and Technology, Dalian University of Technology. He has authored more than 20 research articles on topics in biomedical literature data mining. His current research interests include biomedical literature data mining and information retrieval.



Liang Yang (Member, IEEE) was born in 1986. He received the B.S. degree in computer science and technology from Dalian University of Technology, Dalian, China, in 2009 and the Ph.D. degree in computer application from Dalian University of Technology, in 2016.

He is currently a Lecturer with the School of Computer Science and Technology, Dalian University of Technology, Dalian, China. His research interests include sentiment analysis and opinion mining.



Hongfei Lin (Member, IEEE) received the B.Sc. degree from Northeastern Normal University, Changchun, China, in 1983, the M.Sc. degree from Dalian University of Technology, Dalian, China, in 1992, and the Ph.D. degree from Northeastern University, Shenyang, China, in 2000.

He is currently a Professor with the School of Computer Science and Technology, Dalian University of Technology, where he is also the Director of the Information Retrieval Laboratory.

He has authored more than 100 research papers in various journals, conferences, and books. His research interests include information retrieval, text mining for biomedical literature, biomedical hypothesis generation, information extraction from huge biomedical resources, and learning-to-rank.